94.3
Pass

Shreyansh Jain

PDF generated at: 16 Feb 2026 03:46:03 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 32 min 29 sec on 14 Feb 2026 21:14:32 IST

Candidate Information

Email	shreyanshjain811@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	14 Feb 2026 21:14:32 IST
Time taken	32 min 29 sec/ 90 min
Work Experience	3 years
Invited by	Kalyani
Invited on	13 Feb 2026 19:04:49 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	0%

2	Spring Boot Intermediate	75%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	60%	Hard	0%
Spring	0%	JPA	0%
Spring Data	0%	Transactional	0%
Medium	97%	Implementation	100%
Arrays	100%	Algorithms	100%
Problem Solving	100%	Java	100%
Exceptions	100%	OOP	100%

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	2 min 35 sec	✗
2	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	0/5	-	3 min 27 sec	✗
3	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 41 sec	✓
4	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	54 sec	✓
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 1 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Wind Farms Coding	Problem Solving (Intermediate)	75/75	-	9 min 8 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Vending Machine Coding	Java (Intermediate)	75/75	-	12 min 52 sec	✓

1. JSON Deserialization

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
```

```
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● **@JsonRootName("checker")**
- ○ @JsonName("checker")
- ○ @JsonTypeName("checker")

@JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME)
@JsonTypeName("checker")

No comments.

2. Spring Data Transactional

Incorrect

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
   @Id
   private Long id;
   private String name;
   private Integer courseLength;
   private Integer courseLevel;
   @JsonBackReference
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "student_id")
   private Student student;
   private boolean inProgress;
   //getter & setters
}
```

Student.java

```
@Entity
public class Student {
   @Id
   private Long id;
```

```
    private String name;
    private String surname;
    @JsonManagedReference
    @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
    private List<Lesson> lessons;
    //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods
```

```
    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
```

```
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] It will try to delete the student then the DataIntegrityViolationException will be thrown.

- [x] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".

- [x] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.

- [ ] It will throw a RuntimeException with the message "Student not found".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

3. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }

- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- None of the above

No comments.

---

4. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Configuration class annotated with @EnableGlobalMethodSecurity

- Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)** ✓
- None above

No comments.

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }
- private void manage(HttpSecurity http) {   http.csrf() }
- private void manage(HttpSecurity http) {   http.csrf().enable() }
- ** None of the above. CSRF is configured by default when Spring Security Dependency is added. **

No comments.

6. Wind Farms

Correct

Coding | Implementation | Medium | Arrays | Algorithms | Problem Solving

Question description

An energy company needs to place a control center to manage a distributed wind farm. Your task is to determine the optimal location for the control center that minimizes the total connection cost.

For each wind farm *i*:

- The farm is located at coordinates *(x[i], y[i])*
- The control center is at coordinates *(cx, cy)*
- The distance is calculated as *|x[i] - cx| + |y[i] - cy|*

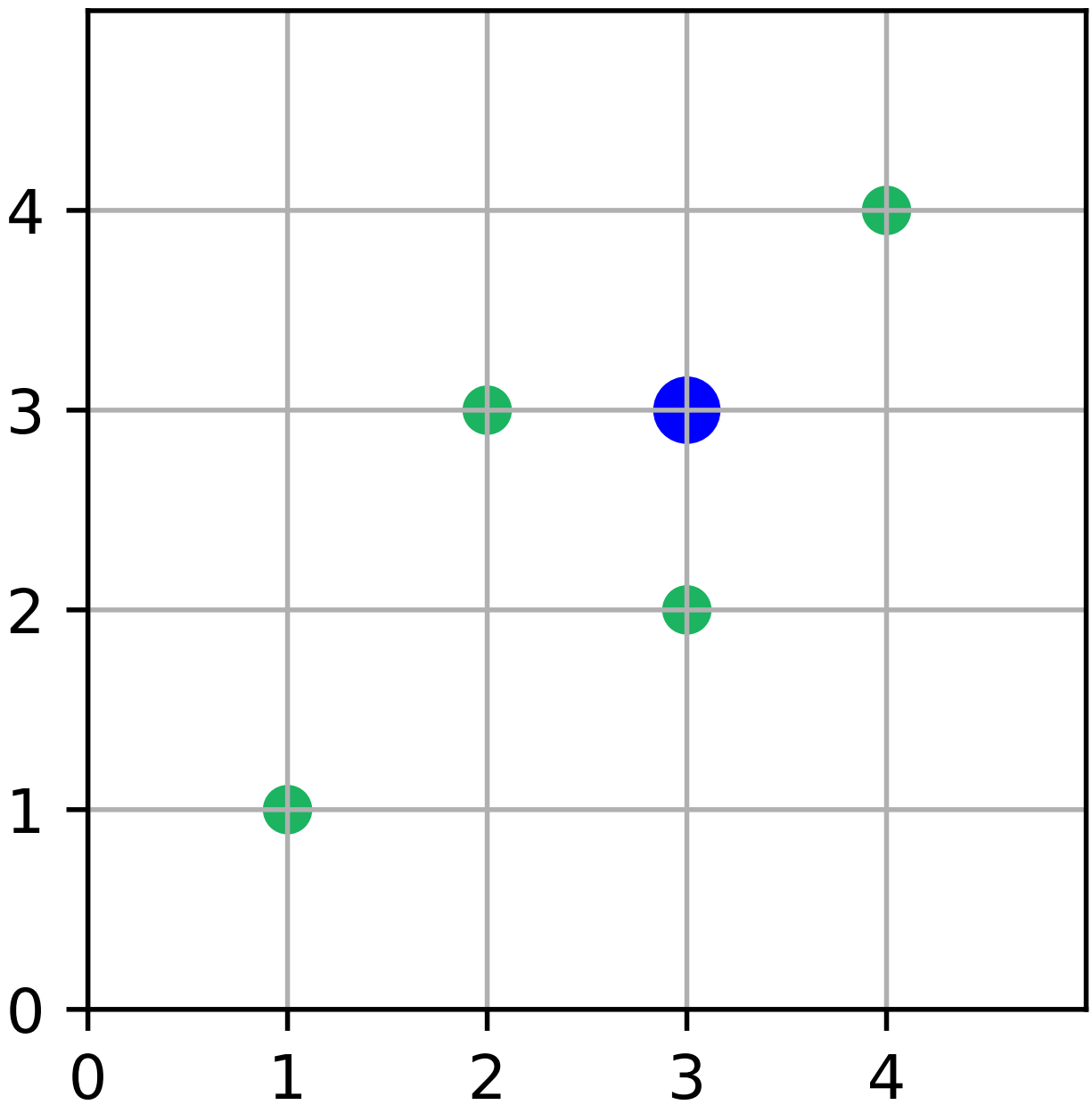
- The connection cost is *premium[i] * distance*

Example

x = [1, 3, 2, 4]

y = [1, 2, 3, 4]

premium = [1, 3, 2, 4]



The optimal location is at position [3, 3], resulting in:

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 3| = 4	1	4 * 1 = 4
1	(3, 2)	|3 - 3| + |2 - 3| = 1	3	1 * 3 = 3
2	(2, 3)	|2 - 3| + |3 - 3| = 1	2	1 * 2 = 2
3	(4, 4)	|4 - 3| + |4 - 3| = 2	4	2 * 4 = 8

The total connection cost is 17, which is the minimum possible.

Function Description

Complete the function *windFarms* in the editor with the following parameter(s):

int premium[n]: the distance multiplier for the cost to connect
int x[n]: *x* coordinates of the wind farms
int y[n]: *y* coordinates of the wind farms

Returns

int: the minimum cost to connect the wind farms

Constraints

- $1 \le n \le 10^3$
- $1 \le x[i], y[i] \le 10^4$
- $1 \le premium[i] \le 50$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the size of the array *premium*.
The next *n* lines each contain an element, *premium[i],* where 0 ≤ i < n.
The next line contains an integer *n*, the size of *x[]*.
The next *n* lines each contain an element, *x[i]*.
The next line contains an integer *n*, the size of *y[]*.
The next *n* lines each contain an element, *y[i]*.

Note: *n* is the same value in each instance.

▼ SAMPLE CASE 0

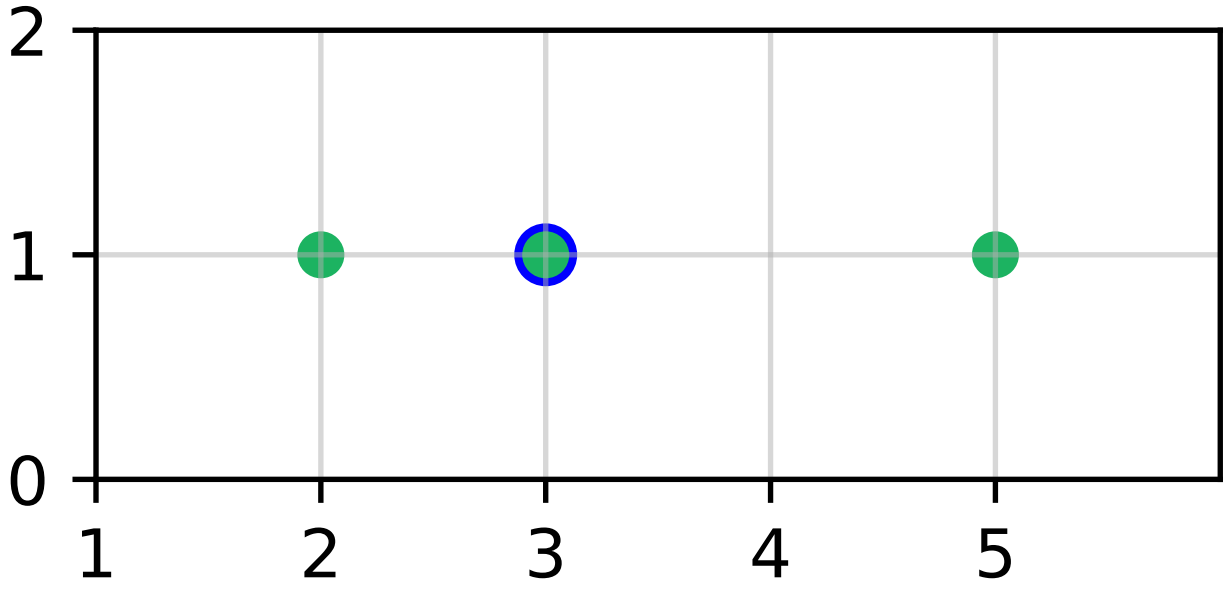
Sample Input 0

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
2    →  x = [2, 3, 5]
3
5
3    →  y[] size n = 3
1    →  y = [1, 1, 1]
1
1
```

Sample Output 0

```
3
```

Explanation 0



Locate the controller at farm 1.

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 1| = 1	1	1 * 1 = 1
1	(3, 1)	|3 - 3| + |1 - 1| = 0	1	0 * 1 = 0

2	(5, 1)	|5 - 3| + |1 - 1| = 2	1	2 * 1 = 2

▼ SAMPLE CASE 1

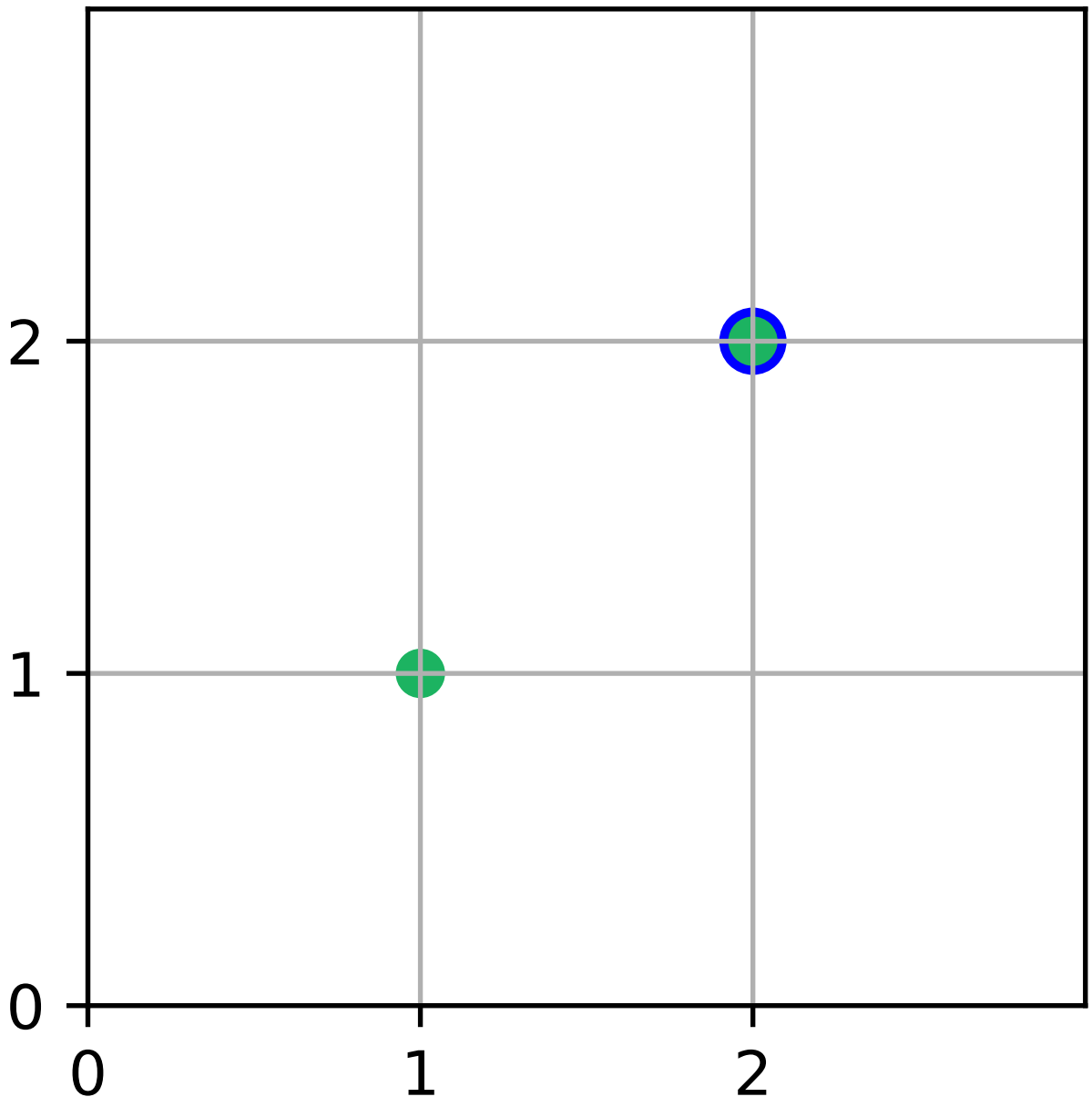
Sample Input 1

```
STDIN    Function
-----    --------
2    →  premium[] size n = 2
1    →  premium = [1, 10]
10
2    →  x[] size n = 2
1    →  x = [1, 2]
2
2    →  y[] size n = 2
1    →  y = [1, 2]
2
```

Sample Output 1

```
2
```

Explanation 1



The sum of the distances to a center at (1, 1), (1, 2), (2, 2), or (2, 1) is 2. Since the premium to connect farm 1 is 10, it is best to locate at position (2, 2) for a cost of 2 * 1 + 0 * 10 = 2.

▼ SAMPLE CASE 2

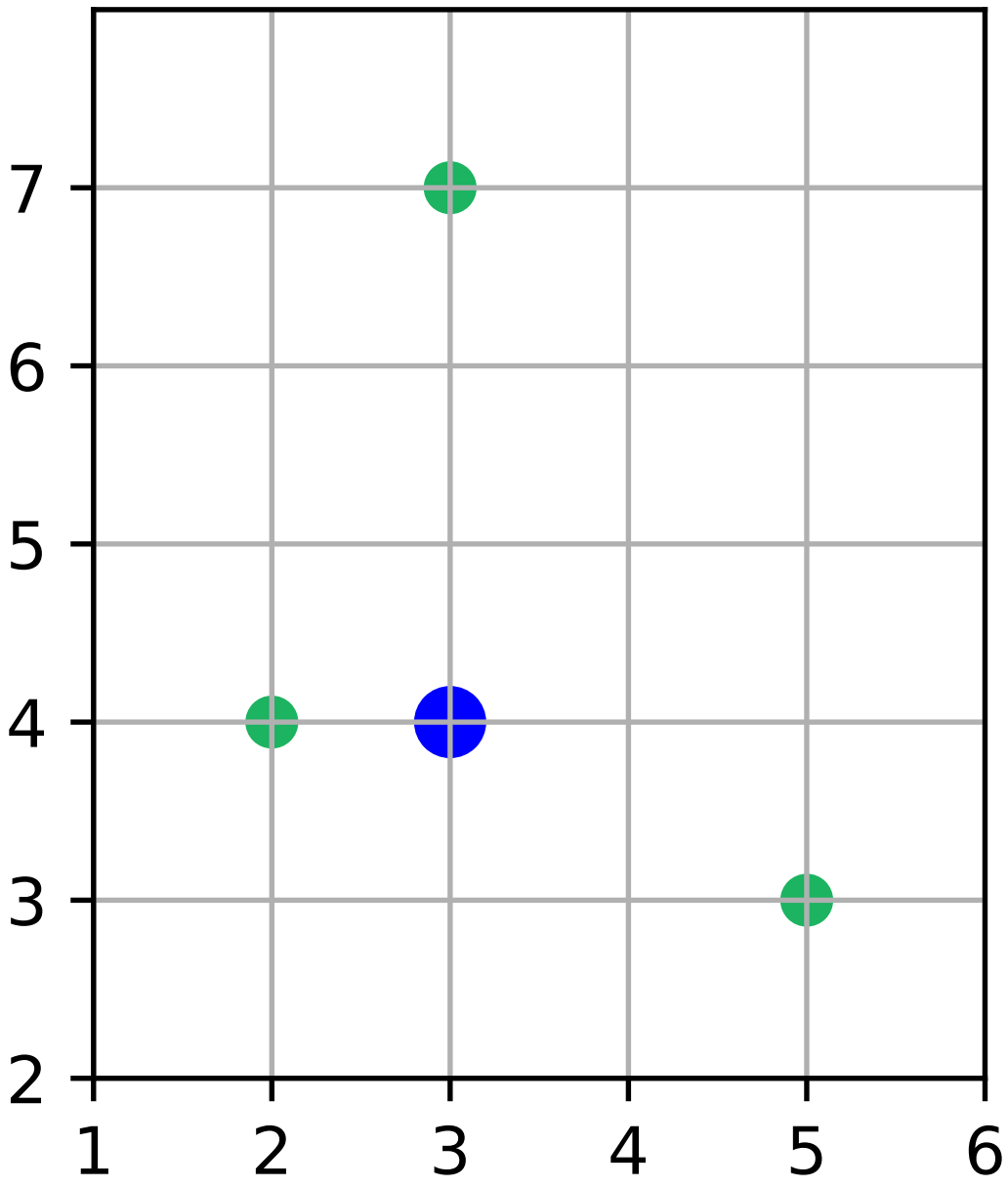
Sample Input 2

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
5    →  x = [5, 2, 3]
2
3
3    →  y[] size n = 3
3    →  y = [3, 4, 7]
4
7
```

Sample Output 2

```
7
```

Explanation 2



The optimal location is (3, 4) for a cost of 3 + 1 + 3 = 7.

Interviewer guidelines

▼ SOLUTION

Skills: Loops and counters, I/O

The cost can be calculated for x and y coordinates separately. The function examines each possible x and y coordinate for the control center separately (from 1 to 10000), computes the total cost of connecting all farms for each possible coordinate, and captures the minimum cost found. The ultimate minimum cost is the summation of these minimum costs for x and y coordinates separately.

Optimal Solution

```
def windFarms(premium, x, y):
    minCostX = 10**10
    n = len(x)

    for cx in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cx - x[i])

        minCostX = min(minCostX, cost)

    minCostY = 10**10
    for cy in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cy - y[i])

        minCostY = min(minCostY, cost)

    return minCostX + minCostY
```

▼ **COMPLEXITY ANALYSIS**

Time Complexity - O(n)

The universe of x and y values are tested against 10000 values.

Space Complexity - O(1)

Constant space is required regardless of n.

Candidate's Solution

Language used: **Java 8**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'windFarms' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY premium
     *  2. INTEGER_ARRAY x
     *  3. INTEGER_ARRAY y
     */

    public static int windFarms(List<Integer> premium, List<Integer> x, List<Integer> y) {
        int cx = weightedMedian(x, premium);
        int cy = weightedMedian(y, premium);

        long totalCost = 0;

        for(int i = 0; i < x.size(); i++) {
            long distance = Math.abs(x.get(i) - cx) + Math.abs(y.get(i) - cy);
            totalCost += distance * premium.get(i);
        }
        return (int)totalCost;
    }

    public static int weightedMedian(List<Integer> coords, List<Integer> weights) {
        int n = coords.size();
```

```java

        List<int[]> pairs = new ArrayList<>();
        long totalWeight = 0;

        for(int i = 0; i < n; i++) {
            pairs.add(new int[] {coords.get(i), weights.get(i)});
            totalWeight += weights.get(i);
        }

        pairs.sort(Comparator.comparingInt(a -> a[0]));

        long cumulative = 0;
        for(int[] p: pairs) {
            cumulative += p[1];
            if(cumulative * 2 >= totalWeight) {
                return p[0];
            }
        }

        return pairs.get(n - 1)[0];
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int premiumCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> premium = IntStream.range(0, premiumCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

```

```
        int xCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> x = IntStream.range(0, xCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int yCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> y = IntStream.range(0, yCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.windFarms(premium, x, y);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.1027 sec	32.1 KB

TestCase 1	Easy	Sample	Success	1	0.1139 sec	32.1 KB
TestCase 2	Easy	Sample	Success	1	0.1087 sec	31.8 KB
TestCase 3	Easy	Sample	Success	2	0.1158 sec	32 KB
TestCase 4	Easy	Hidden	Success	2	0.1167 sec	31.9 KB
TestCase 5	Easy	Hidden	Success	2	0.1131 sec	32 KB
TestCase 6	Medium	Sample	Success	6	0.1228 sec	32 KB
TestCase 7	Medium	Hidden	Success	6	0.1147 sec	31.8 KB
TestCase 8	Medium	Hidden	Success	6	0.1376 sec	32.1 KB
TestCase 9	Hard	Hidden	Success	12	0.1752 sec	35.1 KB
TestCase 10	Hard	Hidden	Success	12	0.2223 sec	35.8 KB
TestCase 11	Hard	Hidden	Success	12	0.1523 sec	35.4 KB
TestCase 12	Hard	Hidden	Success	12	0.1384 sec	35.6 KB

No comments.

7. Java: Vending Machine

Correct

Coding Medium Java Exceptions OOP

Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

Constraints

- $1 \leq totalNumberOfRequests \leq 10^4$
- $0 \leq buttonNumber \leq 2$
- $0 \leq money \leq 100$
- $0 \leq quantity$ available of each $drink \leq 10^4$
- $0 \leq price$ per unit of each $drink \leq 10^4$
- No two drinks are assigned the same *buttonNumber.*

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.
The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.
The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.
The fourth line contains an integer, *totalNumberOfRequests .*
The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*
- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is*

out of stock".

- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*
- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
    HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkRelatedToButton.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
        ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
        serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
        serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();

        if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
            throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
        }

        if(serveDrinkSummary.change < 0){
            throw new InsufficientMoneyException();
        }
        return serveDrinkSummary;
    }
}
```

Candidate's Solution

Language used: **Java 8**

```
1 import java.io.*;
```

```
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}
class VendingMachine{

    private Map<Integer, Drink> drinks = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinks.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)
        throws OutOfStockException, InsufficientMoneyException{

            Drink drink = drinks.get(buttonPressed);

            if(drink.getQuantityLeft() <= 0) {
                throw new OutOfStockException(drink.getName() + " is out of stock");
            }

```

```
            if(money < drink.getPrice()) {
                throw new InsufficientMoneyException();
            }

            int change = money - drink.getPrice();
            ServeDrinkSummary summary = new ServeDrinkSummary();
            summary.servedDrink = drink;
            summary.change = change;
            return summary;
    }
}
class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

```

```
        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

```

```
    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

```

```
    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0497 sec	25.5 KB
TestCase 1	Easy	Hidden	Success	6	0.0654 sec	25.3 KB
TestCase 2	Easy	Hidden	Success	6	0.0612 sec	27.5 KB
TestCase 3	Easy	Hidden	Success	6	0.0801 sec	27.5 KB
TestCase 4	Medium	Hidden	Success	10	0.0726 sec	27.4 KB

TestCase 5	Medium	Hidden	Success	10	0.1056 sec	41.8 KB
TestCase 6	Medium	Hidden	Success	10	0.1037 sec	40.4 KB
TestCase 7	Hard	Hidden	Success	13	0.1093 sec	41.4 KB
TestCase 8	Hard	Hidden	Success	13	0.1027 sec	40.1 KB

No comments.